Faculty of Engineering and Technology

Department of Computer Science

Artificial Intelligence - COMP338

Project Report No.1

Prepared by:

Students Name:	**IDs:**
Rahaf Badwan	1220654
Ayah Abdalmuti	1212074

Instructor: Dr. Mohammad Helal **and** Dr. AbdelRahman Hamrsha

Semester: 2025/2026.

Abstract

The goal of This project used a Genetic Algorithm (GA) to find the hidden 32-bit binary passcode. The passcode was randomly generated by the algorithm itself and was used as the "target." Initially, every GA begins with an initial population of candidate solutions (chromosomes). Every chromosome was evaluated by a fitness function that tallies how many bits matched the target.

In every generation of the GA, the highest fitness-rated chromosomes were selected to create new chromosomes using a combination of crossover (the combination of portions from two parent chromosomes) and mutation. Over subsequent generations, populations improved as the chromosomes became increasingly similar to the target.

Convergence plots were generated to represent how much the best fitness corresponded with the number of generations. The result is that a GA can accurately locate a hidden 32-bit binary passcode; therefore, it can be concluded that GAs have demonstrated their usefulness in solving problems of search and optimization, especially when the available search space is large and when no exact method for solving is available.

Table of Contents

Abstract 2
1. Introduction (Background) 5
2. Problem Definition 6
3. Solution Representation 6
4. Fitness Function 7
5. Genetic Algorithm Steps 8
6. Implementation Explanation 9
6.1 Generate Target Passcode Method 9
6.2 Create Individual and Population Methods 9
6.3 Fitness Function Method 10
6.4 Parent Selection Method 10
6.5 Crossover Method (Single-Point Crossover) 11
6.6 Mutation Method 11
6.7 Genetic Algorithm Main Loop (Evolution Loop) 12
6.8 Saving Convergence Data to CSV 13
6.9 Convergence Plot 13
7. Results 14
7.1 Terminal Output (Summary Results) 14
7.2 Mutation Rate = 0.01 15
7.2 Mutation Rate = 0.02 16
7.3 Mutation Rate = 0.05 17
8. Conclusion 18
9. References 19

Table of Figure

Figure 1:Generate Target Passcode Method 9
Figure 2:Create Individual and Population Methods 9
Figure 3:Fitness Function Method 10
Figure 4:Parent Selection Method 10
Figure 5:Crossover Method 11
Figure 6:Mutation Method 11
Figure 7:Evolution Loop in run_ga Method 12
Figure 8:Saving Convergence Data 13
Figure 9:Convergence Plot Method 13
Figure 10:Terminal Output 14
Figure 11:GA convergence curve for mutation rate = 0.01 15
Figure 12:GA convergence curve for mutation rate = 0.02 16
Figure 13:GA convergence curve for mutation rate = 0.05 17

1. Introduction (Background)

Genetic Algorithms (GAs) are search and optimization methods inspired by natural evolution. They are useful for solving problems where the search space is very large, or where traditional methods cannot easily find a solution [1]. Instead of testing every possible option, a GA starts with a group of random candidate solutions and improves them step by step through several generations.

Each candidate solution is represented by a chromosome, which is a sequence of genes. In this project, each gene represents one bit in a binary passcode. The quality of each chromosome is measured by a fitness function, which shows how close the chromosome is to the desired solution [2]. Chromosomes with higher fitness are more likely to be selected to produce the next generation.

The GA uses three main processes to evolve the population:

1. **Selection:** Choosing the best chromosomes based on their fitness to be parents of the next generation.
2. **Crossover:** Combining parts of two parent chromosomes to create new offspring with mixed characteristics.
3. **Mutation:** Making small random changes to some genes to maintain diversity and avoid getting stuck in a local optimum.

By repeating these steps over many generations, the population gradually improves, and better solutions appear. Genetic Algorithms are widely used because they can efficiently search complex problems and do not require strict mathematical formulas.

In this project, the GA is applied to find a hidden 32-bit binary passcode. The algorithm starts with a random population of candidate solutions and evolves them until one chromosome exactly matches the target passcode. This project demonstrates how GAs can effectively solve search and optimization problems.

2. Problem Definition

In this project the Genetic Algorithm's objective is to discover a randomly generated 32-bit binary passcode.

The Genetic Algorithm has been designed for use with a binary 32-bit password that is created at the beginning of the program. Each password is made up of the numbers 0 and 1.

To reach the goal of finding the password, the Genetic Algorithm has to evolve the population of possible solutions to form a solution that is identical to the correct password based on the number of bits that match with the correct password.

Each candidate solution will be compared against the password by determining how many bits from the candidate solution match with those of the password.

The process of evolution will take place over a number of generations, with the aim of finding the password either once a solution has been found with all bits matching (fitness = 32), or the maximum number of generations has been reached.

3. Solution Representation

In the present study, each solution is represented as a chromosome composed of 32 binary genes, where each gene can take the value of zero (0) or one (1), corresponding to a single digit in the passcode.

Thus, each chromosome represents a potential passcode. The objective of the Genetic Algorithm is to evolve these chromosomes until a chromosome is obtained whose 32-bit sequence exactly matches the target passcode.

4. Fitness Function

In this project, the fitness function measures how close each chromosome is to the target 32-bit passcode. Each chromosome consists of a sequence of 32 binary bits, and its fitness is calculated by counting the number of bits that match the corresponding bits in the target passcode.

The fitness function can be defined as:

Fitness(C) = number of bits in chromosome C that match the target passcode T

Where:
C represents the current chromosome
T represents the target passcode
Ci and Ti denote the bits at position in C and T, respectively

For example, if a chromosome is 10110010... and the target passcode is 11110011..., the fitness function counts the number of matching bits at the same positions. A higher number of matching bits results in a higher fitness value [3].

The fitness function plays a crucial role in guiding the selection process, as chromosomes with higher fitness values are more likely to be selected as parents for the next generation. This process helps the population gradually improve over successive generations until a chromosome exactly matches the target passcode.

5. Genetic Algorithm Steps

The Genetic Algorithm is comprised of a base set of simple and straightforward procedures used to evolve the population toward the correct passcode.

1) Fitness Evaluation

Initially, all chromosomes in the population are assessed using the fitness function. The fitness value provides a count of how many bits aligned with the target passcode. The higher the fitness value of a chromosome, the better solution it represents.

2) Selection

Once chromosomes have been evaluated for fitness, the two chromosomes with the highest fitness values will be used as parents to produce new solutions for the subsequent generation.

3) Crossover

The selected parents will undergo a single-point crossover where a random crossover point is selected. The contents of each parent before and after the crossover point will be combined to produce new offspring. This step facilitates passing on beneficial traits obtained through evolution of the parents into the next generation.

4) Mutation

Following the crossover, the offspring will undergo mutation. Each bit of information will have a (0.02) likelihood of being flipped from a 0 to a 1 or vice versa. Mutation is employed to maintain genetic diversity in the population and avoid the algorithm becoming trapped within a particular local solution.

5) Termination Condition

The cycling of the previously described steps will proceed until the solution equals the desired passcode (fitness=32) or the highest number of generations is achieved

6. Implementation Explanation

The Genetic Algorithm for this project was implemented using the Python programming language.

6.1 Generate Target Passcode Method

```python
# Number of bits in the passcode
BITS = 32

# Generate a random target passcode
def generate_target():
    return [random.randint(0, 1) for _ in range(BITS)]
```

Figure 1:Generate Target Passcode Method

This approach produces the concealed target passcode that the Genetic Algorithm seeks to find

It generates a list of 32 binary digits, with each digit being either 0 or 1.

The target passcode is created randomly at the beginning of each program execution, guaranteeing that each run can yield varied outcomes

6.2 Create Individual and Population Methods

```python
# Number of bits in the passcode
BITS = 32

# Size of the population
POP_SIZE = 150

# Create a random individual (chromosome)
def create_individual():
    return [random.randint(0, 1) for _ in range(BITS)]

# Create the initial population
def create_population(size):
    return [create_individual() for _ in range(size)]
```

Figure 2:Create Individual and Population Methods

This portion of the code creates the first-generation solutions for the Genetic Algorithm.

To start, one chromosome is created. Each chromosome consists of a list of 32 bits (0/1) representing a potential guess of the passcode.

The initial population of 150 chromosomes is then produced from the random creation of several chromosomes. The idea behind this random creation is to ensure that the initial population contains enough variety so that there will be many options to choose from when searching for the "optimal" solution through trial and error.

6.3 Fitness Function Method

```
# Fitness function: count matching bits
def fitness(individual, target):
    return sum(1 for i in range(BITS) if individual[i] == target[i])
```

Figure 3:Fitness Function Method

The fitness function indicates how well an individual chromosome has fitness.

The fitness function checks the chromosome against the passcode that is being searched for and checks the bit positions for the number of matching bits.

If a chromosome has a fitness score of 32, that means the chromosome is a perfect match to the passcode being checked.

6.4 Parent Selection Method

```
# Select the two best individuals as parents
def select_parents(population, target):
    sorted_pop = sorted(population, key=lambda ind: fitness(ind, target), reverse=True)
    return sorted_pop[0], sorted_pop[1]
```

Figure 4:Parent Selection Method

The procedure chooses the two best chromosomes from the population based off of their fitness value where the chromosomes are sorted by descending order of their fitness value; the top two

chromosomes are chosen as the parents. By doing this, it allows the best solutions to be passed onto the next generation of chromosomes.

6.5 Crossover Method (Single-Point Crossover)

```
# Single-point crossover
def crossover(parent1, parent2):
    point = random.randint(1, BITS - 1)
    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]
    return child1, child2
```

Figure 5:Crossover Method

A technique that allows the generation of new offspring through the combined effect of two parent chromosomes. After a randomly chosen crossover point has been selected, bits from both parents,' chromosomes will be exchanged to generate two children. The resultant offspring will enable the beneficial genes of both solutions to be combined.

6.6 Mutation Method

```
# Probability of mutation
MUTATION_RATE = 0.02
# Mutate bits with a given probability
def mutate(individual, rate):
    for i in range(BITS):
        if random.random() < rate:
            individual[i] ^= 1
```

Figure 6:Mutation Method

Random mutations introduce changes to the genetic diversity of the population. A chromosome's bits have a 2% chance of being move from 1 to 0 or vice versa. When an algorithm has mutated bits, it reduces the risk of becoming trapped in a local optimum.

6.7 Genetic Algorithm Main Loop (Evolution Loop)

```python
# Run the Genetic Algorithm
def run_ga(target, mutation_rate=MUTATION_RATE, csv_filename="convergence.csv", verbose=True):
    if verbose:
        print("Target passcode (bits):")
        print(bits_to_string(target))

    population = create_population(POP_SIZE)
    best_fitness_history = []

    start_time = time.time()
    solution_found = False
    generation_found = None

    # Evolution loop
    for gen in range(1, MAX_GENERATIONS + 1):
        fitness_values = [fitness(ind, target) for ind in population]
        best_fit = max(fitness_values)
        best_fitness_history.append(best_fit)

        # Print progress every 100 generations (only if verbose)
        if verbose and (gen % 100 == 0 or best_fit == BITS):
            print(f"Generation {gen}, best fitness = {best_fit}/{BITS}")

        # Stop if solution is found
        if best_fit == BITS:
            solution_found = True
            generation_found = gen
            break

        # Select parents
        parent1, parent2 = select_parents(population, target)

        # Elitism: keep best parents
        new_population = [parent1, parent2]

        # Create new population
        while len(new_population) < POP_SIZE:
            p1, p2 = random.choice(population), random.choice(population)
            child1, child2 = crossover(p1, p2)
            mutate(child1, mutation_rate)
            mutate(child2, mutation_rate)

            new_population.append(child1)
            if len(new_population) < POP_SIZE:
                new_population.append(child2)

        population = new_population

    elapsed_seconds = time.time() - start_time
```

Figure 7:Evolution Loop in run_ga Method

This portion describes how the algorithm evolves over time.

Each Generation records its fitness values for each Generation, identifies the Elite (Best Solution), selects the parent population, and creates a new population via reproduction (Crossover) and genetic variation (Mutation).

The algorithm terminates either when it finds the correct passcode or reaches the maximum number of allowed Generations.

6.8 Saving Convergence Data to CSV

```python
# Save convergence data to CSV
with open("convergence.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["generation", "best_fitness"])
    for i, score in enumerate(best_fitness_history, start=1):
        writer.writerow([i, score])
```

Figure 8:Saving Convergence Data

The best fitness value achieved in each generation of this program is saved as a CSV file for future reference. This file can then be used to assess and visualize the rate of convergence of this algorithm. The file shows how well the algorithm improved over time.

6.9 Convergence Plot

```python
plotting.py > plot_convergence
import matplotlib.pyplot as plt

def plot_convergence(history):
    generations = list(range(1, len(history) + 1))
    plt.plot(generations, history)
    plt.title("Genetic Algorithm Convergence")
    plt.xlabel("Generation")
    plt.ylabel("Best Fitness")
    plt.grid(True)
    plt.show()
```

Figure 9:Convergence Plot Method

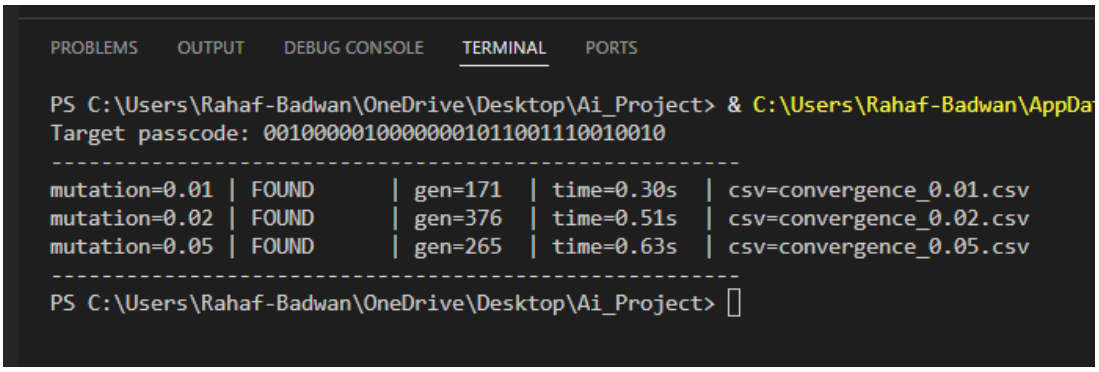
This method generates a graph that shows the best fitness value over generations.

The plot clearly illustrates how the Genetic Algorithm gradually improves until it reaches the optimal solution.

7. Results

The Genetic Algorithm was executed multiple times using different mutation rates (0.01, 0.02, and 0.05) while keeping all other parameters and the target passcode fixed. For each run, the convergence rate was recorded and stored in a CSV file, and the best fitness over generations was visualized.

7.1 Terminal Output (Summary Results)



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Rahaf-Badwan\OneDrive\Desktop\Ai_Project> & C:\Users\Rahaf-Badwan\AppDa
Target passcode: 00100000100000001011001110010010
-------------------------------------------------------
mutation=0.01 | FOUND      | gen=171  | time=0.30s  | csv=convergence_0.01.csv
mutation=0.02 | FOUND      | gen=376  | time=0.51s  | csv=convergence_0.02.csv
mutation=0.05 | FOUND      | gen=265  | time=0.63s  | csv=convergence_0.05.csv
-------------------------------------------------------
PS C:\Users\Rahaf-Badwan\OneDrive\Desktop\Ai_Project>
```

Figure 10:Terminal Output

Figure 10 shows the terminal output summarizing the performance of the Genetic Algorithm for three different mutation rates: 0.01, 0.02, and 0.05.

For each configuration, the output reports whether the optimal solution was found, the number of generations required to reach it, the execution time, and the name of the generated convergence CSV file.

The results indicate that all tested mutation rates successfully found the target passcode. However, the mutation rate of 0.01 achieved the fastest convergence, reaching the optimal solution in 171 generations with an execution time of approximately 0.30 seconds.

In comparison, mutation rates 0.02 and 0.05 required 376 and 265 generations, respectively, with slightly longer execution times.

These results demonstrate the impact of the mutation rate on convergence speed and confirm that lower mutation rates can lead to more efficient performance for this specific problem.

7.2 Mutation Rate = 0.01

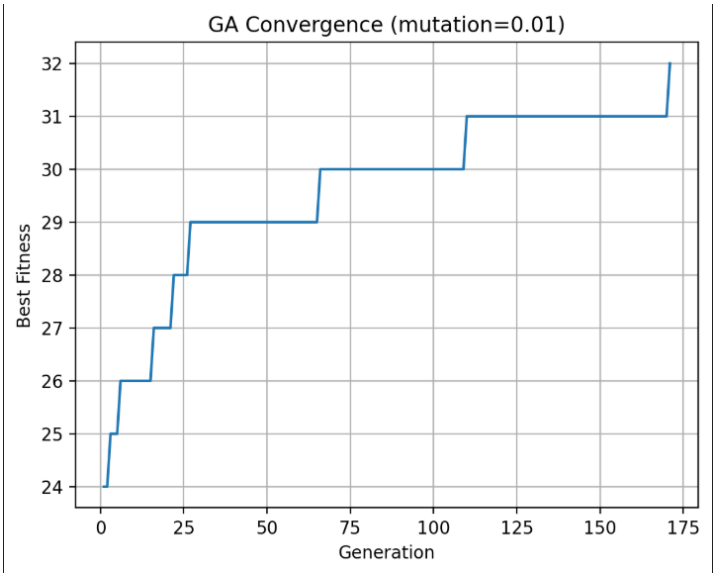


Figure 11:GA convergence curve for mutation rate = 0.01

For a mutation rate of 0.01, the algorithm showed a stable and gradual improvement in fitness over generations.

The convergence curve demonstrates a smooth increase with minimal fluctuations, indicating effective exploitation of high-quality solutions.

The optimal solution was found after 171 generations, which was the fastest among all tested mutation rates.

This result suggests that a low mutation rate was sufficient to preserve good genetic material while still introducing small variations that guided the search toward the global optimum.

7.2 Mutation Rate = 0.02

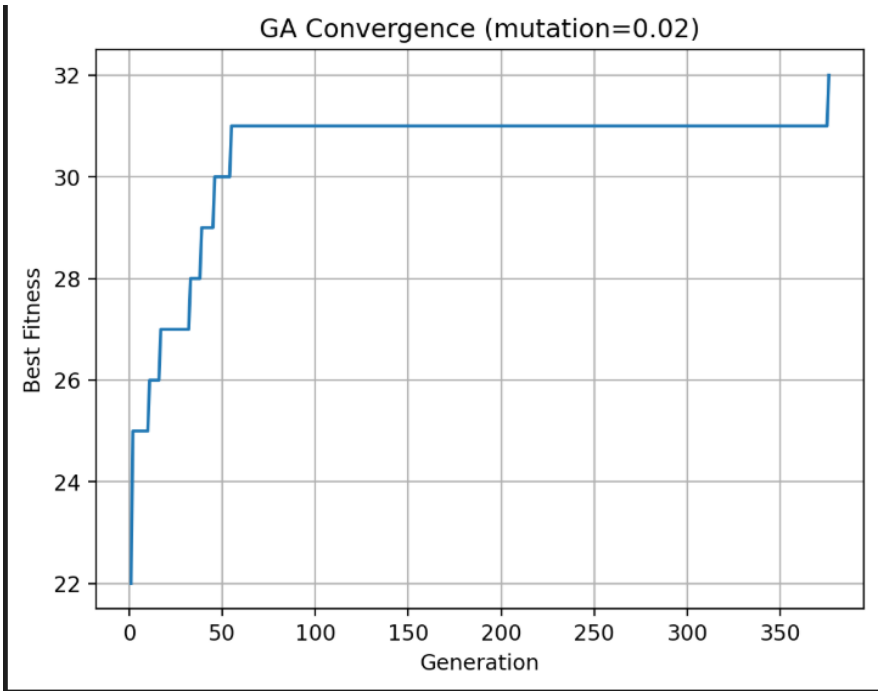


Figure 12:GA convergence curve for mutation rate = 0.02

With a mutation rate of 0.02, the algorithm initially achieved rapid improvements in fitness. However, the convergence curve exhibits a long plateau phase, where no improvement occurred for many generations.

The optimal solution was found after 376 generations, which is significantly slower compared to the 0.01 mutation rate.

This behavior indicates that the increased mutation rate occasionally disrupted promising solutions, slowing down the final convergence despite early progress.

7.3 Mutation Rate = 0.05

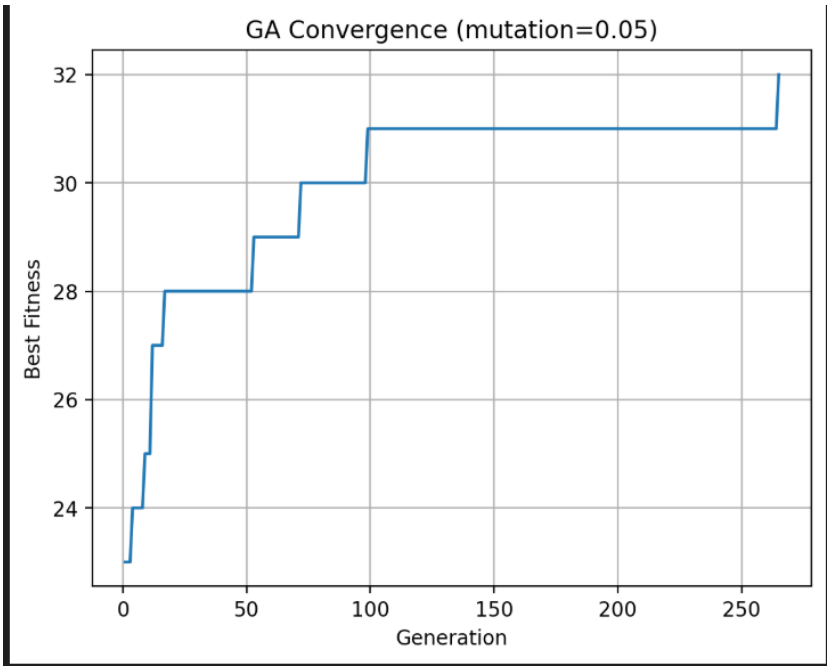


Figure 13:GA convergence curve for mutation rate = 0.05

At a mutation rate of 0.05, the convergence curve shows higher variability and less stability.

Although the algorithm maintained good exploration capabilities, excessive mutation introduced randomness that delayed convergence.

The optimal solution was reached after 265 generations, which is faster than 0.02 but slower than 0.01.

This result highlights the trade-off between exploration and exploitation when using higher

8. Conclusion

This project applied a Genetic Algorithm that successfully identified a concealed 32-bit binary code using the evolutionary strategy approach and accomplished both convergence on the optimum solution as well as producing a fitter result that was equal to 100%. This strongly indicates that Genetic Algorithms are efficient tools for solving complex problems related to both searching and optimizing.

The testing results support that the way in which the individual components of Genetic Algorithms functioned to refine solutions was influenced significantly by the selection, crossover, and mutation processes used. Additionally, there was clear evidence that the population size chosen and the mutation rate applied were critical in determining the rate at which solutions became converged.

Due to the stochastic nature of Genetic Algorithms, the number of generations required to find the final solution will likely vary between individual runs; nonetheless, the behaviour demonstrated by the algorithm in terms of producing reliable, accurate results has been confirmed.

The findings of this project validate that Genetic Algorithms are powerful, adaptive, and versatile methods that can be applied to a variety of large-scale, complex search problems.

9. References

[1] *Goldberg, D. E. Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley, 1989.*

[2] *GeeksforGeeks. "Introduction to Genetic Algorithms."*

[3] *Holland, J. H. Adaptation in Natural and Artificial Systems. University of Michigan Press, 1975.*